

Análisis y evolución de las empresas del S&P 500

Memoria del proyecto de Shell y Entornos Linux

Yassin Pellicer Lamla

13 de septiembre de 2026

Contenidos

1	Objetivo	2
2	Elección del dataset	2
2.1	Formato y estructura	2
2.2	Datos relevantes y preprocesamiento	3
3	Automatización mediante cron	3
4	Scripts de procesamiento	4
4.1	script_delete.sh	4
4.2	script_download.sh	4
4.3	script_sanitize.sh	4
5	Extracción y análisis de datos	4
5.1	Reporte de empresas con mayor capitalización bursátil	6
5.2	Reporte de sectores con mayor capitalización bursátil	6
5.3	Reporte de mayores ganadores por métrica de 52 semanas	6
5.4	Reporte de evolución del mercado	6
5.5	Volcado de resultados a los informes	6
5.5.1	Reportes .txt	7
5.5.2	Reportes .html	7
6	Registro (log)	7
7	Conclusión	8
A	Uso de IA en funciones de ayuda y formateo	9

1 Objetivo

El proyecto automatiza la descarga, depuración, análisis y presentación de información financiera de las compañías del índice S&P 500. Para cada ejecución diaria se conserva una instantánea del conjunto de datos y se generan informes de texto y HTML.

Los informes permiten consultar el ranking de capitalización bursátil, la agrupación por sectores y su ranking, las empresas que más próximas están a su máximo de 52 semanas y la evolución histórica de precio y capitalización.

El objetivo final es disponer de un sistema reproducible y automatizado que permita obtener periódicamente una visión general del estado de las empresas del S&P 500, al mismo tiempo que se aplican herramientas y conceptos propios de Shell y de la administración de sistemas Linux, como la ejecución programada de tareas, el procesamiento de archivos y la generación automática de informes.

2 Elección del dataset

El dataset utilizado en este proyecto consiste en un archivo .csv de 500 entradas correspondientes a los datos de cada una de las 500 empresas más rentables de la bolsa estadounidense. En este caso se utiliza un único dataset en formato CSV publicado por el repositorio abierto *S&P 500 Companies Financials* de la organización *datasets* en GitHub. Se descarga desde:

<https://raw.githubusercontent.com/datasets/s-and-p-500-companies-financials/main/data/constituents-financials.csv>

El repositorio público del dataset es:

<https://github.com/datasets/s-and-p-500-companies-financials>

La fuente es pública y accesible por descarga mediante el comando `curl`. El proyecto no presupone que el archivo sea una fuente de datos en tiempo real, ya que se actualiza una vez al día: simplemente satisface el requisito de actualización mediante una descarga programada diariamente, que guarda una instantánea con fecha propia.

2.1 Formato y estructura

El formato es CSV con una fila de cabecera y una fila por compañía. El script de descarga lo almacena como `datasets/constituents-financials_AAAAMMDD.csv`. Los campos son los siguientes:

Campo	Descripción
Symbol	Símbolo bursátil de la compañía.
Name	Nombre de la compañía.
Sector	Sector o industria asignada.
Price	Precio de la acción.

Campo	Descripción
Price/Earnings	Relación precio-beneficio.
Dividend Yield	Rentabilidad por dividendo.
Earnings/Share	Beneficio por acción.
52 Week Low	Precio mínimo durante las últimas 52 semanas.
52 Week High	Precio máximo durante las últimas 52 semanas.
Market Cap	Capitalización bursátil.
EBITDA	Beneficio antes de intereses, impuestos, depreciaciones y amortizaciones.
Price/Sales	Relación entre el precio y las ventas.
Price/Book	Relación entre el precio de mercado y el valor contable.
SEC Filings	URL de documentación presentada ante la SEC.

2.2 Datos relevantes y preprocesamiento

No todos estos datos son relevantes para el estudio. En el proyecto sólo se consideran los campos `Symbol`, `Name`, `Sector`, `Price`, `52 Week High` y `Market Cap`. El preprocesador invocado en el servicio de descarga automático se deshace de cualquier entrada que no tenga, como mínimo, estas columnas. Sin embargo, las columnas no significativas se conservan de todas maneras para permitir la ampliación y creación de métricas más complejas en un futuro.

3 Automatización mediante cron

El archivo `cron/crontab.txt` define una ejecución diaria a las 12:00 horas

```
0 12 * * * /ruta/al/proyecto/cron/script_cron.sh
```

El script `cron/script_cron.sh` ejecuta las etapas de limpieza, descarga, saneamiento y creación de informes. El procesamiento está separado en scripts Bash con responsabilidades concretas para cada uno:

- `script_delete.sh`: elimina snapshots CSV con más de 30 días.
- `script_download.sh`: descarga el CSV y reintenta la operación hasta diez veces ante un fallo de red.
- `script_sanitizesh`: valida los campos requeridos y normaliza el CSV.
- `script_analisis_csv_txt.sh`: genera los informes de texto.
- `script_analisis_csv_html.sh`: genera el informe HTML a partir de las plantillas localizadas en `scripts/html/`.

Los scripts de análisis aceptan opcionalmente una fecha `AAAAMMDD`; si se omite, emplean la fecha actual. Esto permite regenerar un informe histórico sin volver a descargar el dataset.

4 Scripts de procesamiento

`cron/script_cron.sh` inicia el proceso automatizado por el servicio `cron` y ejecuta los 3 siguientes scripts de procesamiento

- `script_delete.sh`
- `script_download.sh`
- `script_sanitizesh`

Estos scripts se almacenan en la ruta `project/scripts/` del proyecto.

4.1 `script_delete.sh`

Este script borra por defecto los datasets con una antigüedad mayor de 30 días. Concretamente, extrae la fecha del nombre de los archivos y, mediante un bucle, comprueba si el archivo consultado tiene una antigüedad superior a 30 días respecto a la fecha actual. En ese caso, elimina el archivo.

La conservación del histórico de archivos durante 30 días permite comparar e interpretar la evolución de la economía durante un período corto y significativo sin generar reportes excesivamente largos.

4.2 `script_download.sh`

Este script descarga el `.csv` del remoto y le da un nombre especial que le añade la coletilla `_<date>` al final, de manera que el `.csv` descargado el 11/09/2026 se llamaría `constituentes-financials_20260911.csv`.

El script es a prueba de fallos de conexión desde el cliente. Es decir, si por algún motivo el equipo pierde la red, el programa reintentará la descarga hasta 10 veces en un plazo de 6 minutos. También controla las descargas vacías o la ausencia del recurso a descargar. Los datasets se almacenan en la ruta `project/datasets` del proyecto.

4.3 `script_sanitizesh`

Este script toma el dataset recién descargado y comprueba que su estructura es válida para la extracción y análisis de datos posterior.

En primer lugar elimina todas aquellas entradas del `.csv` que carezcan de las columnas `Symbol`, `Name`, `Sector`, `Price`, `52 Week High` y `Market Cap`. A continuación reformatea los campos `Name` y `Sector` para eliminar las `,` embebidas entre comillas `'''` que dificultan la lectura del archivo (ya que el archivo utiliza la `,` como símbolo ortográfico en nombres como "Nike, Inc.", lo que impide la correcta separación de los campos entre comas).

5 Extracción y análisis de datos

Una vez ejecutados satisfactoriamente los tres primeros scripts de procesamiento, se ejecutan los dos scripts correspondientes a la extracción y análisis de los datos del dataset:

- `script_analisis_csv_txt.sh`
- `script_analisis_csv_html.sh`

Estos scripts se almacenan en la ruta `project/scripts/` del proyecto.

Ambos scripts realizan los mismos análisis sobre el dataset (y con los mismos algoritmos), pero generan los resultados en formatos diferentes.

El script `script_analisis_csv_txt.sh` genera cuatro archivos de texto, cuyos nombres siguen el formato indicado a continuación:

- `highest_grossing_sectors_<date>.txt`
- `highest_winners_52_week_metric_<date>.txt`
- `price_marketcap_evolution_<date>.txt`
- `top_market_cap_companies_<date>.txt`

En todos los casos, `<date>` corresponde a la fecha del día en que se generan los archivos.

Por otro lado, el script `script_analisis_csv_html.sh` genera un informe en formato HTML compuesto por los siguientes archivos:

- `report_<date>.html`
- `script.js`
- `style.css`

Los reportes `.txt` se almacenan en la ruta `project/analysis/<date>` donde `<date>` es la fecha del reporte en cuestión. El HTML generado se anida en una carpeta adicional llamada `report`. Esta es la estructura de los archivos de reporte.

```
project/
├─ analysis/
│   └─ <date>/
│       ├── highest_grossing_sectors_<date>.txt
│       ├── highest_winners_52_week_metric_<date>.txt
│       ├── price_marketcap_evolution_<date>.txt
│       ├── top_market_cap_companies_<date>.txt
│       └─ report/
│           ├── report_<date>.html
│           ├── script.js
│           └─ style.css
```

Se ha preferido separar los reportes de texto en 4 ficheros distintos para hacer la lectura y consulta de la información más sencilla, ya que cada reporte tiene un número de columnas variable y longitudes distintas.

5.1 Reporte de empresas con mayor capitalización bursátil

Este reporte se genera ordenando la columna de capitalización bursátil (`Market Cap`) en orden descendiente por medio del siguiente flujo:

1. Se lee el fichero excluyendo la cabecera mediante el comando `tail`.
2. `sort` ordena las entradas por la columna `Market Cap` en orden descendente.

5.2 Reporte de sectores con mayor capitalización bursátil

Este reporte agrupa las empresas según su sector y calcula la capitalización bursátil total de cada uno mediante el siguiente flujo:

1. Se lee el fichero sin la cabecera mediante `tail`.
2. `sort` ordena las entradas por la columna `Market Cap` en orden descendente.
3. `awk` agrupa las empresas por sector y suma sus valores de `Market Cap`.
4. Los sectores se ordenan de forma descendente según su capitalización total mediante un algoritmo de ordenación en `awk`.

5.3 Reporte de mayores ganadores por métrica de 52 semanas

Este reporte identifica las empresas cuyo precio está más próximo al máximo de las últimas 52 semanas:

1. Se lee el fichero sin la cabecera mediante `tail`.
2. `awk` calcula la distancia porcentual entre `Price` y `52 Week High`.
3. `sort` ordena las empresas de menor a mayor distancia.
4. `cut` elimina el campo auxiliar donde se calculó esta distancia.

5.4 Reporte de evolución del mercado

Este reporte muestra la evolución histórica del precio y de la capitalización bursátil de las empresas ordenados por capitalización media durante el período:

1. Se recorren los datasets disponibles hasta la fecha seleccionada y mediante `awk` se vuelca cada registro por día en la salida.
2. La salida anterior se le pasa a `awk`, que calcula la capitalización media de la empresa durante el período analizado y se la añade a cada fila suministrada.
3. `sort` ordena la entrada por prioridad, primero por `capitalizacion media`, (para conseguir una ordenación), luego por `Symbol`, y por último por fecha.
4. Un último `awk` agrupa las empresas mostrando su evolución.

5.5 Volcado de resultados a los informes

El volcado de la información generada en los informes se hace de manera especial en ambos scripts de análisis.

5.5.1 Reportes .txt

En estos reportes la información se estructura tabularmente y se adapta al formato .txt mediante columnas y cabeceras formadas por caracteres ASCII para facilitar la lectura.

```
+=====+
| HIGHEST GROSSING SECTORS |
+=====+
| Date: 13/09/2026 | Processed entries: 485 |
+=====+
| Semiconductors |
+=====+
| Symbol | Company | Market Cap |
+-----+-----+-----+
| NVDA | Nvidia | 5,271,048,421,376 |
| AVGO | Broadcom | 1,728,006,324,224 |
| MU | Micron Technology | 1,101,452,017,664 |
| AMD | Advanced Micro Devices | 842,569,351,168 |
```

5.5.2 Reportes .html

En estos reportes `script_analisis_csv_html.sh` prepara la estructura HTML que sustituir en el `template.html` mediante el comando `sed`, y mediante el cual construir el informe final.

```
...
{
    row_count++
    printf "        <tr>\n"
    printf "            <td class=\"row-number\">%d</td>\n", row_count
    printf "            <td>%s</td>\n", commas($10)
    printf "            <td>%s</td>\n", fit($1, 8)
    printf "            <td>%s</td>\n", fit($2, 30)
    printf "        </tr>\n"
}
' > rows.html
sed -i '/<!-- TOP MARKET CAP ROWS -->/{
    r rows.html
    d
}' "$HTML_REPORT"
```

6 Registro (log)

El archivo `log.txt` reúne los mensajes de las ejecuciones. Se registran el inicio y fin de las etapas, los archivos utilizados, los reintentos de descarga y las filas descartadas durante el saneamiento. La función `log` añade fecha y hora con milisegundos; el operador `>>` conserva las entradas anteriores:

```
log() {
    echo "[$(date +"%Y/%m/%d %H:%M:%S.%3N")] - $" >> "$LOG"
}
log "DAILY AUTOMATION STARTED"
```

Los mensajes de detalle incluyen la hora y etiquetas como [ok] para operaciones correctas y [ko] para fallos. La función `run_step` comprueba el estado de salida de cada script: si falla, registra la etapa afectada y termina la ejecución con `exit 1`.

Para comandos como `sort`, se redirige la salida de error (descriptor 2) a una función que registra cada mensaje junto al nombre del comando:

```
log_command_error() {
    local tool="$1" error
    while IFS= read -r error; do
        log_detail "[ko] [ERROR] [$tool] - $error"
    done
}
# Fragmento de una tubería:
sort -t',' -k10,10nr 2> >(log_command_error "sort") |
```

Este mecanismo permite localizar errores y conocer cuándo ocurrieron.

7 Conclusión

Este proyecto permite seguir la evolución de las empresas del S&P 500 y convertir datos financieros dispersos en información organizada y fácil de consultar. Su propósito es facilitar la comparación entre empresas y sectores, identificar cuáles concentran mayor capitalización y observar cómo cambian sus precios a lo largo del tiempo.

En lo formativo se ha podido demostrar la utilidad de comandos de `shellscript` aplicados a la extracción de patrones comunes provenientes de una fuente de datos de manera automática mediante `awk`, `grep`, `sort`, `tail`, `sed`, y utilidades de automatización como `cron`.

El proyecto deja frentes abiertos y margen de mejora. Como el procesamiento de las fuentes no ha borrado las columnas no utilizadas en un futuro se podrían extender los scripts de análisis para extraer más características de estas columnas.

A Uso de IA en funciones de ayuda y formateo

Se ha utilizado asistencia de IA como apoyo al desarrollo en las siguientes tareas:

- Elaboración de las funciones auxiliares `commas` y `fit` utilizadas en `awk`, para añadir separadores de miles y limitar la longitud de los textos, respectivamente.
- Desarrollo del algoritmo de ordenación descendente de los sectores según su capitalización bursátil total.
- Formateo de los informes `.txt` mediante tablas de caracteres ASCII, alineación de columnas, cabeceras y presentación de diferencias de precio y capitalización.
- Generación de estructuras HTML para las tablas de los informes, definición de estilos CSS e incorporación de numeración y totales.
- Revisión de fragmentos de Bash y `awk`, incluyendo campos auxiliares de ordenación y condiciones para mostrar registros de evolución.
- Extensivamente en la generación de tablas y gráficos en el fichero HTML. Generación de Javascript, estilos y parseo de HTML.